Web Service Film BGS

Documentazione Tecnica

Bacci Guerrini Salcioli

Anno Scolastico 2025/2026

Contents

1 Introduzione **3**
1.1 Descrizione del progetto . . . 3
1.2 Motivazione . . . 3
1.3 Stack tecnologico . . . 3

2 Architettura del sistema **4**
2.1 Struttura del progetto . . . 4
2.2 Flusso delle richieste . . . 4
2.3 Configurazione del database . . . 4

3 API esterna: imdbapi.dev **4**

4 Endpoint del Web Service **5**
4.1 `GET /api/film/titles` . . . 5
4.1.1 Descrizione . . . 5
4.1.2 Parametri . . . 5
4.1.3 Struttura della risposta (`200 OK`) . . . 5
4.1.4 Logica di funzionamento . . . 5
4.1.5 Codici di errore . . . 5
4.2 `GET /api/film/actors` . . . 6
4.2.1 Descrizione . . . 6
4.2.2 Parametri . . . 6
4.2.3 Struttura della risposta (`200 OK`) . . . 6
4.2.4 Logica di funzionamento . . . 6
4.2.5 Codici di errore . . . 6
4.3 `GET|POST /api/film/oscar` . . . 6

4.3.1 Descrizione 6
4.3.2 Parametri — `GET` 7
4.3.3 Corpo della richiesta — `POST` 7
4.3.4 Struttura della risposta 7
4.3.5 Logica di funzionamento 7
4.3.6 Codici di errore 7

5 Database **8**
5.1 Schema della tabella `oscar` 8
5.2 Dati iniziali 8

6 Router principale **8**

7 Riepilogo degli endpoint **9**

8 Considerazioni sulla sicurezza **9**

1 Introduzione

1.1 Descrizione del progetto

Il progetto **Web Service Film BGS** e' una REST API sviluppata in PHP che offre un livello di astrazione sopra i dati cinematografici pubblici. Il servizio espone tre endpoint che consentono di:

1. cercare film per titolo e ottenerne le informazioni essenziali (titolo, anno, immagine, valutazione);
2. recuperare il cast principale di un film con dettagli anagrafici sugli attori;
3. consultare e inserire i premi Oscar associati ai film, attingendo a un database locale.

1.2 Motivazione

Le API pubbliche gratuite disponibili per i dati cinematografici presentano due limitazioni principali:

- non offrono un endpoint diretto per ricercare gli attori a partire dal titolo di un film;
- restituiscono un volume eccessivo di campi quando si effettua una ricerca per titolo, rendendo necessaria una fase di filtraggio lato client.

Il Web Service Film BGS risolve entrambi i problemi aggregando i dati di piu' chiamate all'API esterna imdbapi.dev e mantenendo in locale un database relazionale dedicato ai premi Oscar.

1.3 Stack tecnologico

Componente	**Tecnologia**
Linguaggio back-end	PHP
Server HTTP	Apache
Database	MySQL
Formato risposta	JSON
API esterna	imdbapi.dev

2 Architettura del sistema

2.1 Struttura del progetto

I file del nostro progetto sono così strutturati: `index.php` funge da entry point e router centrale; la cartella `config/` contiene il file `db.php` per la connessione al database; la cartella `api/film/` ospita i tre handler `titles.php`, `actors.php` e `oscar.php`; infine i file `tabella.sql` e `oscar.sql` definiscono lo schema e i dati iniziali del database.

2.2 Flusso delle richieste

Tutte le richieste HTTP transitano attraverso `index.php`, che funge da router centralizzato. Il flusso e' il seguente:

1. Il client invia una richiesta HTTP (`GET` o `POST`).
2. `index.php` legge il parametro *`url`* dalla query string e verifica il metodo HTTP; restituisce `405 Method Not Allowed` se il metodo non e' consentito.
3. Il router seleziona il file handler corrispondente tramite uno `switch` sul valore di *`url`*.
4. L'handler esegue la logica di business (chiamate all'API esterna o query al database) e scrive la risposta in formato JSON sullo standard output.

2.3 Configurazione del database

La connessione al database e' centralizzata nella funzione `getDb()` definita in `config/db.php`. Essa stabilisce una connessione MySQLi verso il database `oscar_db` sull'host locale, imposta il charset UTF-8 e termina l'esecuzione con un messaggio di errore JSON in caso di connessione fallita.

3 API esterna: imdbapi.dev

Il servizio si appoggia a tre endpoint dell'API pubblica imdbapi.dev:

Endpoint esterno	Scopo
`GET /search/titles?query={titolo}`	Ricerca film per titolo; restituisce un array `titles`.
`GET /titles/{idFilm}`	Dettagli del film per ID; include il campo `stars`.
`GET /names/{idAttore}`	Dettagli anagrafici di una persona (attore/regista).

4 Endpoint del Web Service

4.1 `GET /api/film/titles`

4.1.1 Descrizione

Ricerca film per titolo e restituisce un elenco filtrato ai soli elementi di tipo `movie`, con i campi essenziali: identificatore, titolo, anno di uscita, URL dell'immagine di copertina e valutazione aggregata.

4.1.2 Parametri

Nome	Tipo	Obbligatorio	Descrizione
film	`string`	Si'	Titolo (parziale o completo) del film

4.1.3 Struttura della risposta (`200 OK`)

La risposta e' un oggetto JSON con chiave `film` contenente un array di oggetti. Ogni oggetto include i campi `id`, `title`, `year`, `image` e `rating`.

4.1.4 Logica di funzionamento

L'handler codifica il parametro *film* con `urlencode()` e lo usa per interrogare l'endpoint di ricerca di `imdbapi.dev` tramite cURL. Dalla risposta vengono selezionati solo i risultati con `type = "movie"`; se nessun risultato supera il filtro viene restituito un errore `404`.

4.1.5 Codici di errore

Codice HTTP	Causa
`400 Bad Request`	Parametro *film* assente.
`404 Not Found`	Nessun film di tipo `movie` trovato.
`502 Bad Gateway`	Impossibile contattare `imdbapi.dev`.

4.2 GET /api/film/actors

4.2.1 Descrizione

Ricerca il primo film corrispondente al titolo fornito e restituisce le informazioni anagrafiche degli attori principali (`stars`).

4.2.2 Parametri

Nome	Tipo	Obbligatorio	Descrizione
film	`string`	Si'	Titolo del film

4.2.3 Struttura della risposta (`200 OK`)

La risposta e' un oggetto JSON con la chiave `film` (titolo del film trovato) e la chiave `actors`, un array di oggetti contenenti i campi `name`, `birthday` (in formato `YYYY-MM-DD`), `image` e `birthLocation`.

4.2.4 Logica di funzionamento

L'handler esegue tre chiamate sequenziali all'API esterna:

1. ricerca del film per titolo per ottenere l'ID del primo risultato di tipo `movie`;
2. richiesta dei dettagli del film tramite l'ID, per recuperare la lista degli attori principali (`stars`);
3. per ciascun attore, richiesta dei dati anagrafici tramite l'ID persona.

4.2.5 Codici di errore

Codice HTTP	Causa
`400 Bad Request`	Parametro *film* assente.
`404 Not Found`	Nessun film di tipo `movie` trovato.
`502 Bad Gateway`	Impossibile contattare `imdbapi.dev`.

4.3 GET|POST /api/film/oscar

4.3.1 Descrizione

Questo endpoint gestisce due operazioni distinte a seconda del metodo HTTP:

- `GET`: ricerca nel database locale i premi Oscar associati a un film.
- `POST`: inserisce un nuovo record Oscar nel database.

4.3.2 Parametri — GET

Nome	Tipo	Obbligatorio	Descrizione
film	string	Si'	Titolo (parziale o completo) del film

4.3.3 Corpo della richiesta — POST

Campo	Tipo	Obbligatorio	Descrizione
title	string	Si'	Titolo del film
year	integer	Si'	Anno del premio
category	string	Si'	Categoria del premio
winner	string	Si'	Nome del vincitore

4.3.4 Struttura della risposta

Per il metodo GET (200 OK), la risposta contiene la chiave oscar con un array di record ordinati per anno decrescente, ciascuno con i campi id, title, year, category e winner.

Per il metodo POST (201 Created), la risposta contiene un oggetto con il campo message che conferma l'avvenuta creazione del record.

4.3.5 Logica di funzionamento

Per il GET, la ricerca avviene tramite una query con operatore LIKE sul campo titolo, usando un prepared statement per prevenire attacchi SQL Injection. I risultati vengono restituiti ordinati per anno decrescente.

Per il POST, il body JSON viene decodificato e i quattro campi obbligatori vengono validati prima di eseguire un INSERT con prepared statement.

4.3.6 Codici di errore

Codice HTTP	Causa
400 Bad Request	Parametro *film* assente (GET), oppure campi mancanti o JSON non valido (POST).
405 Method Not Allowed	Metodo HTTP diverso da GET e POST.
500 Internal Server Error	Errore di esecuzione della query SQL.

5 Database

5.1 Schema della tabella `oscar`

La tabella `oscar` e' definita nel file `tabella.sql` e presenta la seguente struttura:

Colonna	Tipo	Vincolo	Descrizione
`id`	`INT`	PK, AUTO_INCREMENT	Identificatore univoco del record
`title`	`VARCHAR(255)`	NOT NULL	Titolo del film
`year`	`YEAR`	NOT NULL	Anno della cerimonia
`category`	`VARCHAR(255)`	NOT NULL	Categoria del premio Oscar
`winner`	`VARCHAR(255)`	NOT NULL	Nome del vincitore o del film vincitore

5.2 Dati iniziali

Il nostro db (oscar.sql) contiene circa 100 record relativi alle cerimonie degli Academy Awards dal 1972 al 2023. Le categorie coperte sono: Miglior Film, Miglior Regia, Miglior Attore, Miglior Attrice, Miglior Attore Non Protagonista, Miglior Attrice Non Protagonista, Miglior Sceneggiatura, Miglior Colonna Sonora, Miglior Film Animazione, Miglior Fotografia, Miglior Montaggio, Miglior Effetti Visivi, Miglior Costumi e Miglior Canzone.

6 Router principale

Il file `index.php` e' il punto di ingresso unico del servizio. Imposta gli header di risposta (`Content-Type: application/json` e `Access-Control-Allow-Origin: *`) e verifica che il metodo HTTP sia `GET` o `POST`, restituendo `405` in caso contrario. In base al valore del parametro *`url`*, delega l'esecuzione al file handler corrispondente tramite `require`; se il percorso non corrisponde a nessun endpoint noto, restituisce `404`. ront-end ospitate su origini diverse.

7 Riepilogo degli endpoint

Metodo	Percorso	Parametro	Risposta
`GET`	`/api/film/titles`	*?film={titolo}*	Lista film: id, titolo, anno, immagine, rating
`GET`	`/api/film/actors`	*?film={titolo}*	Titolo film e lista attori (nome, nascita, foto, luogo)
`GET`	`/api/film/oscar`	*?film={titolo}*	Lista premi Oscar del film
`POST`	`/api/film/oscar`	Body JSON	Conferma creazione (`201`)

8 Considerazioni sulla sicurezza

- Tutte le query al database utilizzano **prepared statements** con binding dei parametri tramite `mysqli_stmt_bind_param`, prevenendo attacchi di tipo SQL Injection.
- L'endpoint con metodo POST permette a chiunque di aggiungere premi oscar al nostro db, senza alcun controllo, quindi potenzialmente le informazioni contenute potrebbero diventare non del tutto corrette.